

IMRAN'S SUMMER 2026

NLP / LLM Engineering Roadmap — v3

12 weeks · 7+ hrs/day · \$0 API spend · Real job-brief grounded · Production-ready

What's in this document:

- Zero-cost API stack (no credit card, no spend) wired into every week from day 1.
- Weeks 1-3: foundations built by doing — every day ends with something working, not just something read.
- Weeks 4-9: 3 projects built against real Upwork/Fiverr job briefs, each with an eval harness.
- Production-readiness skills woven in: structured output, cost control, prompt injection defense, FastAPI, multi-provider model swapping, messy document handling.
- Week 10: 3-day n8n + WhatsApp Business API sprint — the fastest local Pakistani freelance path to a first real paying client.
- Weeks 10-12: portfolio, listings, direct outreach, internship applications.
- Rest cadence: 1 full day off every 2 weeks (6 total). Rest days are gold-highlighted in every table.

YOUR \$0 API STACK

Confirmed: you can run the entire 12-week plan for \$0.

Research confirmed: between Google AI Studio, Groq, Cerebras, and OpenRouter, you can run real production workloads — chatbots, agents, RAG pipelines — for \$0/month in 2026. No credit card required for any of the providers below.

Sign up for ALL of these before Week 1 Day 1 — takes about 30 minutes total.

1. Google AI Studio — your primary LLM (best free tier in 2026)

- URL: aistudio.google.com — email only, no credit card
- Model: Gemini 2.0 Flash — free tier gives 15 RPM, 1 million tokens per context window
- Also gives you free embedding models for your RAG pipeline — one key for both LLM and embeddings
- LangChain install: `pip install langchain-google-genai`
- ```
from langchain_google_genai import ChatGoogleGenerativeAI
```
- ```
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash")
```
- Privacy note: Google uses prompts to improve models. Fine for learning projects. Do not feed real client data through it — switch to a paid tier when you have a real paying client.

2. Groq — your speed fallback (fastest free inference available)

- URL: console.groq.com — email only, no credit card
- Model: Llama 3.3 70B at ~320 tokens/second — use when you need fast responses (voice agent, real-time chat)
- Rate limits: 1,000 requests/day on free tier (reduced from 14,400 in earlier 2026 — verify current limits at signup)
- Fully OpenAI-SDK-compatible — one line change in LangChain:
- ```
from langchain_groq import ChatGroq
```
- ```
llm = ChatGroq(model="llama-3.3-70b-versatile")
```

3. OpenRouter — your variety router and rate-limit escape hatch

- URL: openrouter.ai — no credit card for free models
- Single API key routes to 20+ free models — when Gemini or Groq rate-limits you, OpenRouter keeps you moving
- Free models include Llama, Mistral, Gemma, and others — enough for all learning and project work
- OpenAI-SDK-compatible: set `base_url` to `https://openrouter.ai/api/v1`

4. Cerebras — your high-volume backup

- URL: cloud.cerebras.ai — no credit card
- ~1M tokens/day on Llama 3.3 70B — use for batch eval runs where you need volume

5. ChromaDB — your vector database (fully local, always free)

- `pip install chromadb` — runs entirely on your laptop, no account, no API, no cost
- Used for all RAG work throughout Weeks 2-9

6. LangSmith — observability (free tier)

- URL: smith.langchain.com — free tier sufficient for all project-level monitoring
- Shows you exactly what your agent did step-by-step — real production monitoring skill

7. Streamlit Cloud — deployment (free, same as PawGress)

- No change from your existing workflow — deploy all 3 projects here

8. n8n — automation (free self-hosted)

- Run locally during the Week 10 sprint: `npx n8n` (no account, no cost)
- Or use their free cloud tier for testing

When does \$0 break down?

Only one scenario: a real paying client whose documents are sensitive (medical, legal, financial). At that point, you'd need a paid API to avoid your client's data going through Google's training pipeline. But by that point you have income to cover it — a \$20-30 API bill on a PKR 50,000+ gig is irrelevant. Until then, \$0 all the way.

THE PLAN AT A GLANCE

End-of-summer target

3 shipped LLM/RAG/agent projects built against real client-style briefs, each with a basic eval harness and production-readiness touches, at least one real local paid client via the n8n/WhatsApp sprint, a polished portfolio, and outreach in motion for freelance work and/or internships. Total API spend: \$0.

Phase 1 — Foundations (Weeks 1-3)

Tokens, embeddings, prompting, vector DBs, LangChain, LangGraph — structured output and basic LLM security woven in from day 1. Every single day ends with something built or tested, not just read.

Phase 2 — Build against real briefs (Weeks 4-9)

- Project 1 (Weeks 4-5): RAG knowledge-base chatbot, scoped against a real Upwork/Fiverr brief.
- Project 2 (Weeks 6-7): LangGraph agent with tool use, scoped against a real automation/agent brief.
- Project 3 (Weeks 8-9): Deepen the stronger project using the upgrade real postings most commonly ask for.

Phase 3 — Ship and monetize (Weeks 10-12)

n8n + WhatsApp local-client sprint, portfolio, Upwork/Fiverr listings, Pakistan-specific direct outreach, internship applications.

PRE-WEEK SETUP (do this before Day 1)

Do all of this before your first study day — takes 1-2 hours total. Starting Day 1 without these accounts will slow you down.

Day	Focus	Tasks
Setup	API accounts	☐ Sign up at aistudio.google.com → get your Gemini API key ☐ Sign up at console.groq.com → get your Groq API key ☐ Sign up at openrouter.ai → get your OpenRouter API key ☐ Sign up at cloud.cerebras.ai → get your Cerebras API key ☐ Sign up at smith.langchain.com → get your LangSmith API key ☐ Store all 5 keys in a .env file: GOOGLE_API_KEY, GROQ_API_KEY, OPENROUTER_API_KEY, CEREBRAS_API_KEY, LANGCHAIN_API_KEY
Setup	Python environment	☐ Create a virtual environment: <code>python -m venv llm_env && source llm_env/bin/activate</code> ☐ <code>pip install langchain langchain-google-genai langchain-groq langchain-community langgraph chromadb python-dotenv streamlit fastapi uvicorn pydantic openai</code> ☐ Test: <code>from langchain_google_genai import ChatGoogleGenerativeAI</code> — if it imports without error, you're ready ☐ Create a GitHub repo called <code>summer-2026-llm</code>, push an initial commit with just the <code>.gitignore</code> and <code>README</code>
Setup	CampusX playlist	☐ Bookmark: youtube.com/playlist?list=PLKnIA16_RmvYsvB8qkUQuJmJNuiCUJFPL ☐ Do NOT start watching yet — open specific videos only when the relevant week arrives. Watching ahead without building alongside = passive learning trap.

LLM fundamentals, APIs, tokenization, structured output

Phase 1 · Foundations

Done when:

You can call Gemini and Groq from scratch (no copy-paste), explain why one prompt beats another, reliably force structured JSON output via tool-calling, and have shipped 3 small CLI tools that prove it. API cost: \$0.

CampusX playlist: Videos 1-3 this week.

Watch Video 1 (Generative AI vs Agentic AI) on Day 1 after reading. Watch Video 2 and 3 (LangChain vs LangGraph) on Day 2. Watch same day you code, never ahead.

Day	Focus	Tasks
Day 1	Tokens & context	☐ Study: what a token is, context window limits, temperature/top_p/max_tokens ☐ Try the OpenAI tokenizer visualizer on sample text to see tokens visually ☐ Watch CampusX Video 1 (Generative AI vs Agentic AI) after reading ☐ Write a 1-page README in your own words — your personal reference, not copied notes
Day 2	System vs user prompts	☐ Study: difference between system and user prompts, why system prompts persist behavior ☐ Skim Anthropic's prompt engineering docs at docs.anthropic.com (still the best prompting reference even though you're using Gemini) ☐ Watch CampusX Videos 2-3 (LangChain vs LangGraph overview) ☐ Manually write and test 5 system prompts for different personas in Google AI Studio web UI before writing any code
Day 3	API setup + first call	☐ Load all 5 API keys from .env using python-dotenv ☐ Write your first Gemini API call via LangChain: one message, print the response ☐ Repeat with Groq: change one line (ChatGoogleGenerativeAI → ChatGroq), confirm same output shape ☐ This proves your zero-cost provider-swap setup works before you build anything real on top of it
Day 4	Prompting + structured output (core skill)	☐ Study: few-shot prompting, chain-of-thought prompting ☐ Study in depth: structured output via tool-calling — real job postings list this as a named core skill, not a bonus ☐ Practice: define a Pydantic model, use with_structured_output to force Gemini to return it, test on 5 varied messy inputs

Day	Focus	Tasks
		☐ Confirm it never breaks the schema shape — this is the difference between a demo and something a client can integrate
Day 5	Embeddings + basic LLM security	☐ Study: what an embedding is, cosine similarity, why text becomes a vector ☐ Generate embeddings for 5 sentences using Google's free embedding model (same API key) ☐ New: study basic prompt injection — what ignore-previous-instructions attacks look like, simple defenses (input sanitization, system-prompt hardening, output validation)
Day 6	Build: Tool 1 + Tool 2	☐ Tool 1: Explain-it-simply CLI — topic + level (5yo/undergrad/expert), system-prompt driven, using Gemini ☐ Tool 2: Streaming chat CLI — token-by-token streaming via Groq (fast), maintains message history across turns, /reset and /quit ☐ Test Tool 2 with a 5-turn conversation, confirm turn 5 references turn 2 — proves message history works
Day 7	Build: Tool 3 + push	☐ Tool 3: Structured extractor — Pydantic model, tool-calling, extracts action-items/owner/deadline from messy meeting notes ☐ Test on 3 inputs: clean, messy, and zero-action-item text — confirm it returns an empty list, not a hallucinated fake item ☐ Push all 3 tools to GitHub with a README. Light review — no new material.

Embeddings, vector DBs, RAG theory, messy documents

Phase 1 · Foundations

Done when:

You understand why naive RAG fails, can explain chunking trade-offs, can handle a messy real-world document (not just a clean PDF), and have a working chat-with-a-PDF using ChromaDB and Gemini embeddings. API cost: \$0.

Day	Focus	Tasks
Day 1	ChromaDB setup	☐ pip install chromadb — runs fully locally, no account, no cost ☐ Load 10 sample text snippets, embed them using Google's free embedding model, store in Chroma, run a similarity query ☐ Compare against plain keyword search on the same data — note where semantic search wins
Day 2	Indexing & providers	☐ Study: HNSW vs flat index at a high level — enough to reason about trade-offs ☐ Study: when ChromaDB (local) is enough vs Pinecone/Qdrant/pgvector (production scale) ☐ Load 50-100 chunks, test query speed and quality ☐ Also test: swap Groq in as your LLM for generation to confirm your provider-swap pattern works at this stage
Day 3	Chunking strategies	☐ Study: fixed-size vs semantic vs recursive chunking, why naive chunking loses context ☐ Take a real PDF (a UCP lecture note or article), chunk it 2 different ways, manually inspect which chunks read better
Day 4	Retrieval quality + messy documents	☐ Study: precision/recall applied to retrieval, reranking, hybrid search basics ☐ Handle a messy real-world format: a scanned/low-quality PDF needing OCR, or an Excel/Word file — real client docs are rarely clean PDFs ☐ pip install pytesseract or python-docx depending on which format you pick
Day 5	Evals intro	☐ Study: what makes a RAG answer good vs bad — groundedness, relevance, completeness ☐ Write 10 test questions with expected answers for a small document set ☐ This is your first eval harness seed — the skill real job postings explicitly screen for
Day 6	Build: chat-with-a-PDF (part 1)	☐ Load a real PDF, chunk it, embed with Google's free embedding model, store in ChromaDB ☐ Write the retrieval function: given a question, fetch top-k most relevant chunks ☐ Manually inspect retrieved chunks for 3 test questions

Day	Focus	Tasks
		before wiring generation
Day 7	Build: chat-with-a-PDF (part 2)	☐ Wire retrieved chunks into a Gemini prompt, generate an answer, print it with the source chunk shown ☐ Run your 10 test questions, note which ones fail and why ☐ Push to GitHub with README. Light review.

LangChain, LangGraph, agentic RAG, market research

Phase 1 · Foundations

Done when:

You can build a basic agentic RAG loop (router → retrieve → grade → retry-or-answer) without copying a tutorial, and you've surveyed real job postings so Projects 1 and 2 build against actual client asks. API cost: \$0.

CampusX playlist: Videos 4-10 this week.

Open the relevant video only when you reach that day's topic. Watch → build the same day.

Day	Focus	Tasks
Day 1	LangChain basics	☐ Study <code>init_chat_model</code> , the <code>@tool</code> decorator, how LangChain wraps your free providers ☐ Confirm: ChatGoogleGenerativeAI and ChatGroq both work identically through LangChain's interface — one codebase, swappable providers
Day 2	<code>create_agent</code>	☐ Study <code>create_agent</code> — LangChain's high-level agent entry point ☐ Build a tiny agent with 2 fake tools using Gemini, watch it decide when to call each ☐ Rebuild the same agent using Groq instead — confirm one-line swap works
Day 3	LangGraph state	☐ Study LangGraph's mental model: a state machine vs LangChain's linear chains ☐ Study <code>GraphState</code> (<code>TypedDict</code> tracking question, retrieved docs, generation, <code>retry_count</code>) ☐ Build a minimal 2-node graph: retrieve → generate
Day 4	Conditional edges	☐ Study conditional edges: how the graph decides which node to go to next ☐ Add a grader node that checks if retrieved docs are relevant ☐ Wire a conditional edge: irrelevant docs loop back to a query-rewrite node instead of generating
Day 5	Router + grader + market research	☐ Study the full agentic RAG pattern: router → retrieve → grade → generate-or-retry → hallucination check, with <code>retry_count</code> guard ☐ MARKET RESEARCH (2-3 hours): survey 15-20 real Upwork/Fiverr postings for RAG chatbot, AI agent, LangChain developer. Note: required tools, deliverables, price ranges, how they describe production quality. This grounds Projects 1 and 2 in reality.
Day 6	Build: agentic RAG loop (part 1)	☐ Combine router, retriever (ChromaDB + Gemini embeddings), grader, generator in one LangGraph graph ☐ Use OpenRouter as your LLM here to prove you can swap

Day	Focus	Tasks
		providers mid-project — same code, different key
Day 7	Build: agentic RAG loop (part 2) + REST	☐ Test a deliberately bad retrieval case — confirm the grader catches it and triggers retry instead of a bad answer ☐ Push to GitHub with a graph structure diagram in the README ☐ CHECKPOINT: explain out loud why a retrieval failed and how you'd fix it — if you can't, revisit Days 3-5 before moving to Week 4

REST DAY — after Week 3 (first rest day of the summer). Full day off. No tasks, no videos.

rest

Project 1 — RAG knowledge-base chatbot (real brief)

Phase 2 · Build

Done when:

A deployed RAG app fulfilling an actual real-world job posting's requirements — citations, reliability, eval harness with before/after scores, a specific domain. Built to a standard a real client would accept. API cost: \$0.

Real market signal:

Live Upwork postings for this category include lines like DO NOT APPLY IF YOU HAVE NOT BUILT A PRODUCTION-LEVEL RAG CHATBOT BEFORE. Top profiles list: structured outputs, hybrid retrieval, vector DBs, and production hardening as standard. Build to that bar.

Use the real posting you found in Week 3 Day 5 research as your spec. If it's filled, find the closest current analog.

Day	Focus	Tasks
Wk4 D1	Pick brief + gather docs	☐ From your Week 3 market research, pick one real RAG chatbot posting as your spec ☐ Write one sentence: what this app answers, for whom, using what document set ☐ Gather 5-15 real documents matching that domain
Wk4 D2-3	Ingestion pipeline	☐ Build ingestion: load → chunk → embed (Google free embeddings) → store in ChromaDB, with metadata (source, page) for citations ☐ Handle at least 2 file types — include one messy format you practiced in Week 2
Wk4 D4-5	Retrieval + citations + structured output	☐ Build retrieval with reranking if time allows; generation must cite which source chunk(s) it used ☐ Apply Week 1's structured output skill: force consistent response shape (answer + sources + confidence) via tool-calling ☐ Test manually with 10 questions, inspect citation accuracy by hand
Wk4 D6	Streamlit UI + cost logging	☐ Build the Streamlit interface: upload or preset docs, ask question, see answer + cited sources ☐ Log token count per query using LangSmith (already set up from pre-week setup) — real observability from day 1 of your first project ☐ Swap Groq in for one test run to confirm provider independence
Wk4 D7	Light catch-up	☐ No new material. Catch up on anything incomplete from Days 1-6.
Wk5 D1-2	Eval harness	☐ Write 20 test questions with expected answers and

Day	Focus	Tasks
		expected source documents for your dataset ☐ Write a script that runs all 20 through your app and scores: did it retrieve the right source? did the answer match expected content? ☐ Log results in a table: question, expected, actual, pass/fail, token count
Wk5 D3	Fix failures	☐ Diagnose every failed eval case at the root cause: bad chunking, bad retrieval, or bad prompt ☐ Fix, re-run the suite, document before/after score — this comparison is itself a strong portfolio artifact
Wk5 D4	Edge cases + security	☐ Test empty/no-answer cases: must say I don't know, not hallucinate ☐ Test prompt injection: type ignore previous instructions into the query field — does it break? Add the defense from Week 1. ☐ Test what happens when you swap Cerebras in as the LLM — does your code break or handle it gracefully?
Wk5 D5	Deploy + README	☐ Deploy to Streamlit Cloud (same workflow as PawGress) ☐ Write a client-facing README: the brief it satisfies, architecture, eval score before and after fixes, screenshot or demo GIF, tech stack
Wk5 D6	REST DAY	☐ Second rest day of the summer. Full day off.
Wk5 D7	Real user + retro	☐ Show it to one real person outside this process, note anything that breaks under real usage ☐ Short written retro: what worked, what to change for Project 2

Project 2 — LangGraph agent with tool use (real brief)

Phase 2 · Build

Done when:

An agent that takes real actions against a real brief, with retry logic, LangSmith observability, cost-per-run tracking, a FastAPI endpoint, and an eval harness scoring task success. API cost: \$0.

Day	Focus	Tasks
Wk6 D1	Pick brief + tools	☐ Pick a real AI agent / automation / support-agent posting from your Week 3 research as your spec ☐ Common real patterns found: lead qualification, support escalation, CRM/Slack-connected agents ☐ Write the agent's job in one sentence, list 3-4 concrete tools it needs
Wk6 D2-3	Build the tools	☐ Write each tool as a clean Python function with <code>@tool</code> decorator ☐ Mock external APIs if needed (e.g. a fake order-status lookup function) ☐ Test each tool standalone before wiring into the agent
Wk6 D4-5	Build the LangGraph graph	☐ Design the state machine: router → tool-calling nodes → grader/checker → respond-or-retry ☐ Add <code>retry_count</code> guard and max-step limit so the agent cannot loop forever ☐ Wire LangSmith tracing so every tool call and decision is logged and visible — real postings list observability/monitoring as a standard deliverable
Wk6 D6	Test edge cases	☐ Test tool failure and unexpected data — does the agent recover or crash? ☐ Test an ambiguous request — does it ask a clarifying question or guess badly?
Wk6 D7	Light catch-up	☐ No new material. Catch up if behind.
Wk7 D1-2	Eval harness for the agent	☐ Write 15-20 test scenarios: input → expected tool calls → expected outcome ☐ Score: did the agent call the right tools in the right order? Did it reach the correct final outcome? ☐ Agent evals check the trajectory, not just the final text — harder than RAG evals, more impressive
Wk7 D3	Fix + re-run	☐ Diagnose and fix failed scenarios (common causes: unclear tool descriptions, missing guardrails) ☐ Re-run, document improvement
Wk7 D4	Cost tracking + FastAPI wrapper	☐ Log token usage per run using LangSmith, estimate cost per conversation (at Gemini free-tier pricing: \$0, but log it anyway — clients will ask this when you switch to paid)

Day	Focus	Tasks
		models) ☐ Wrap the agent's core logic behind a FastAPI endpoint — pip install fastapi uvicorn, write one route that accepts a user message and returns the agent's response ☐ This single addition makes your project pluggable into any existing system, not just a Streamlit demo — real client briefs often need this
Wk7 D5	Demo interface	☐ Wrap the agent in a Streamlit chat interface on top of the FastAPI core ☐ Test calling the FastAPI endpoint directly with curl to confirm it works without the UI
Wk7 D6	REST DAY	☐ Third rest day of the summer. Full day off.
Wk7 D7	Deploy + README + retro	☐ Deploy/package cleanly, write the README: brief satisfied, architecture, eval results, cost-per-run, LangSmith trace screenshot ☐ Short retro before choosing Project 3's direction

Project 3 — go deep, market-directed

Phase 2 · Build

Done when:

Your strongest project has the upgrade real postings most commonly ask for beyond basics — not guessed from a generic menu, but based on patterns you actually saw in your Week 3 research. API cost: \$0.

Day	Focus	Tasks
Wk8 D1	Choose direction	☐ Pick whichever of Project 1 or 2 had better traction or excites you more ☐ Pick ONE upgrade based on what you saw repeated across real postings: most commonly hybrid search + reranking, multi-document handling, or multi-provider model support (client already has Azure/other credits) ☐ Write the upgrade in one sentence before starting — scope creep kills projects
Wk8 D2-4	Build the upgrade (part 1)	☐ Hybrid search (BM25 + vector): pip install rank-bm25, combine scores, add a reranker model ☐ OR multi-agent orchestration: a planner agent delegating to sub-agents via LangGraph ☐ OR provider-agnostic swap: rewrite the same project to run across Gemini, Groq, and Cerebras via a config flag — demonstrates you're not locked to one provider, a real client concern
Wk8 D5-6	Build the upgrade (part 2)	☐ Integrate with your existing eval harness to measure objective improvement ☐ Compare before/after eval scores — this is itself a strong case study artifact
Wk8 D7	Light catch-up	☐ No new material. Catch up if behind.
Wk9 D1-3	Production polish	☐ Proper error handling, logging, clean config (no hardcoded keys or paths) ☐ Basic monitoring view if feasible: eval score dashboard or usage log summary ☐ Tighten the UI — this is your most-demoed project
Wk9 D4	Case study writeup	☐ Write a proper case study README: the real brief pattern it answers, architecture diagram, eval methodology, before/after results, what you'd do with more time ☐ This is the single artifact most likely to be read by a recruiter or client — give it real care
Wk9 D5	Find a real user or client	☐ Reach out to 5-10 people or businesses who could plausibly use it, even free in exchange for a testimonial ☐ Record a short demo video or GIF for the README
Wk9 D6	REST DAY	☐ Fourth rest day of the summer. Full day off before Phase 3

Day	Focus	Tasks
		begins.
Wk9 D7	Final fixes + transition	☐ Fix what real usage reveals ☐ Prep mentally: Phase 3 is outward-facing work, not building — different kind of energy needed

n8n sprint + WhatsApp + portfolio

Phase 3 · Ship and monetize

Why this 3-day sprint exists:

Research confirmed: the fastest-converting local Pakistani freelance pattern right now is n8n/Make.com-built WhatsApp AI agents for local SMEs — clinics, tutoring centres, restaurants, salons. One student earned PKR 255,000 in 90 days from one client this way. n8n takes days, not weeks, to learn since it's visual not code. This sprint gets you your first local paid client faster than anything else in this plan.

Done when:

A working n8n + WhatsApp Business API bot for one realistic local SME, a portfolio presenting all 3 core projects, updated resume and LinkedIn. API cost: \$0 (n8n self-hosted, WhatsApp free tier).

Day	Focus	Tasks
Day 1	n8n + WhatsApp setup	☐ Run n8n locally: <code>npx n8n</code> (no account, no cost) ☐ Register a WhatsApp Business API test number via Meta's Cloud API (free, not your personal number) ☐ Follow n8n's official WhatsApp + OpenAI integration docs — use OpenRouter's free endpoint as your LLM since it's OpenAI-SDK-compatible ☐ Get a basic hello-world bot replying to a WhatsApp message before end of day
Day 2	Build one real local-SME bot	☐ Pick one realistic local business type: tutoring centre, clinic, small shop ☐ Build a bot handling 3-4 real intents: pricing, hours, location, human handoff ☐ Connect to a simple knowledge source (Google Sheet or small doc) so it answers from real info, not hallucination ☐ Test thoroughly before moving to outreach
Day 3	Polish + find real leads	☐ Fix obvious breaks, test edge cases ☐ Identify 5-10 real local businesses near UCP or in your network who could use this ☐ Prepare a short pitch: what it does, what problem it solves for them, what you'd charge
Day 4-5	Portfolio site or GitHub profile	☐ Build a portfolio (site or polished GitHub profile README) listing all 3 core projects plus the n8n bot ☐ Each project gets: one-line pitch, tech stack, eval results, live demo link ☐ The eval results are what make this stand out — most student portfolios have none
Day 6	Resume + LinkedIn update	☐ Update resume with this summer's projects using outcome-driven bullets (the same pattern from your earlier resume)

Day	Focus	Tasks
		rebuild) ☐ Add to skills: LangChain, LangGraph, RAG, ChromaDB, evals, FastAPI, LangSmith, n8n ☐ Update LinkedIn headline and summary toward NLP/LLM engineering
Day 7	REST DAY	☐ Fifth rest day of the summer. Full day off.

Freelance push + internship applications

Phase 3 · Ship and monetize

Done when:

Live freelance listings on at least 2 platforms, real outreach sent via both global and Pakistan-specific channels, internship applications out.

Day	Focus	Tasks
Day 1	Freelance listings	☐ Create Upwork listing: RAG chatbot for business documents — price low (\$50-100 fixed) for first reviews ☐ Create Fiverr listing: WhatsApp AI agent for your business — local pricing in PKR to attract Pakistan clients ☐ Both listings: mention your eval harness and cite your eval score — almost nobody does this, instant differentiator
Day 2-3	Direct outreach, two channels	☐ Global: apply to 10-15 real Upwork/Fiverr postings matching your projects — personalized proposals, not a template ☐ Local Pakistan: message the SME leads from Week 10 n8n sprint, plus Lahore-based startups, UCP adjacent businesses — local trust converts faster than global platforms for a first gig ☐ Track responses in a simple spreadsheet: who, when, what was sent, current status
Day 4	Internship applications	☐ Apply to remote AI/NLP internships and relevant local roles (CLE, Force Cloud, and similar) with updated resume and portfolio link ☐ Tailor each application summary line to the specific role — the eval results and FastAPI deployment are your strongest differentiators
Day 5	Follow-ups + prep	☐ Follow up on non-responses from Day 2-3 outreach ☐ Prepare to explain your eval methodology and cost-per-conversation if asked in a call — these are real differentiators in this market ☐ Practice: explain why your RAG eval works, what your retry logic guards against, what the LangSmith trace shows
Day 6	Write 2-3 short posts	☐ Write a short LinkedIn or blog post on each project: what I built, what I learned, what broke ☐ Keep it specific and honest, not hype — specific posts get read, generic ones don't
Day 7	REST DAY	☐ Sixth and final scheduled rest day. Full day off.

Close, deliver, set up the next 6 months

Phase 3 · Ship and monetize

Done when:

Any landed work delivered well, a concrete plan for continuing part-time alongside next semester, and a written retro you can re-read before semester starts.

Day	Focus	Tasks
Day 1-3	Deliver landed work	☐ Deliver any landed gig properly — over-communicate, meet deadlines, ask for a review or testimonial when done ☐ If nothing landed yet: keep outreach going. Most freelance timelines run longer than one summer — this is normal, not failure
Day 4	Document everything	☐ Update portfolio with any new results, testimonials, or numbers ☐ Write a personal retro: what you built, what surprised you, what's weakest going into next semester, what you'd do differently
Day 5	Plan the next 6 months	☐ Decide: continue freelancing part-time during semester, focus on internship search, or both ☐ Pick 1-2 specific things to keep learning during semester without overloading coursework — fine-tuning depth, MLOps basics, or a 4th project are all viable
Day 6	Properly disconnect	☐ Genuine rest before semester restarts — not light review, actual disconnect ☐ You've put in 500+ hours this summer. That deserves a real break.
Day 7	Reset for semester	☐ Light prep for coursework — re-read your retro before classes start ☐ Keep your GitHub active during semester: even one small commit per week maintains momentum

GROUND RULES

- Rest cadence: 1 full day off every 2 weeks, gold-highlighted in every table above. If 2+ days in a row show zero real progress — same bug, same paragraph, can't move forward — take an unscheduled half-day regardless of schedule. Output quality dropping silently is more expensive than a rest day.
- Projects 1 and 2 use the real posting you found in Week 3 Day 5 as their spec. Don't drift into building something no real client actually asked for.
- Never skip the eval harness steps. Real job postings explicitly screen for this. It's not optional polish.
- The n8n WhatsApp sprint is a fast local-cash side-track, not a replacement for the coded LangChain/LangGraph spine. Don't let it eat into your Phase 2 build weeks.

- **CampusX playlist: open specific videos only when the relevant week arrives. Watch → build the same day. Never watch ahead without coding alongside.**
- **If you fall behind, compress scope before you compress rest or API budget. Cut a feature, not your judgment.**
- **When you land a real paying client who needs their documents handled privately: that is the moment to add a paid API. By then you have income. Until then, \$0 all the way.**